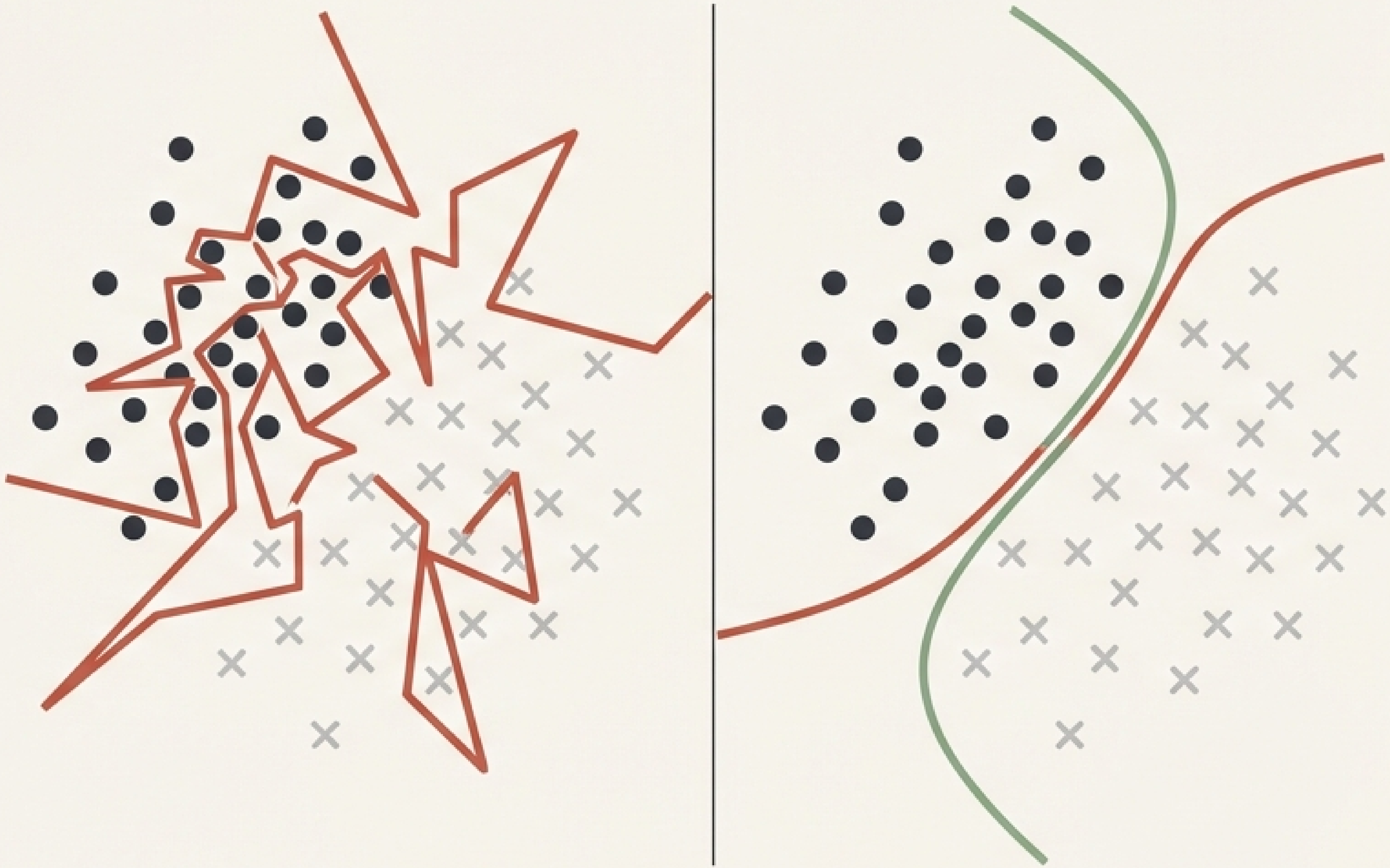
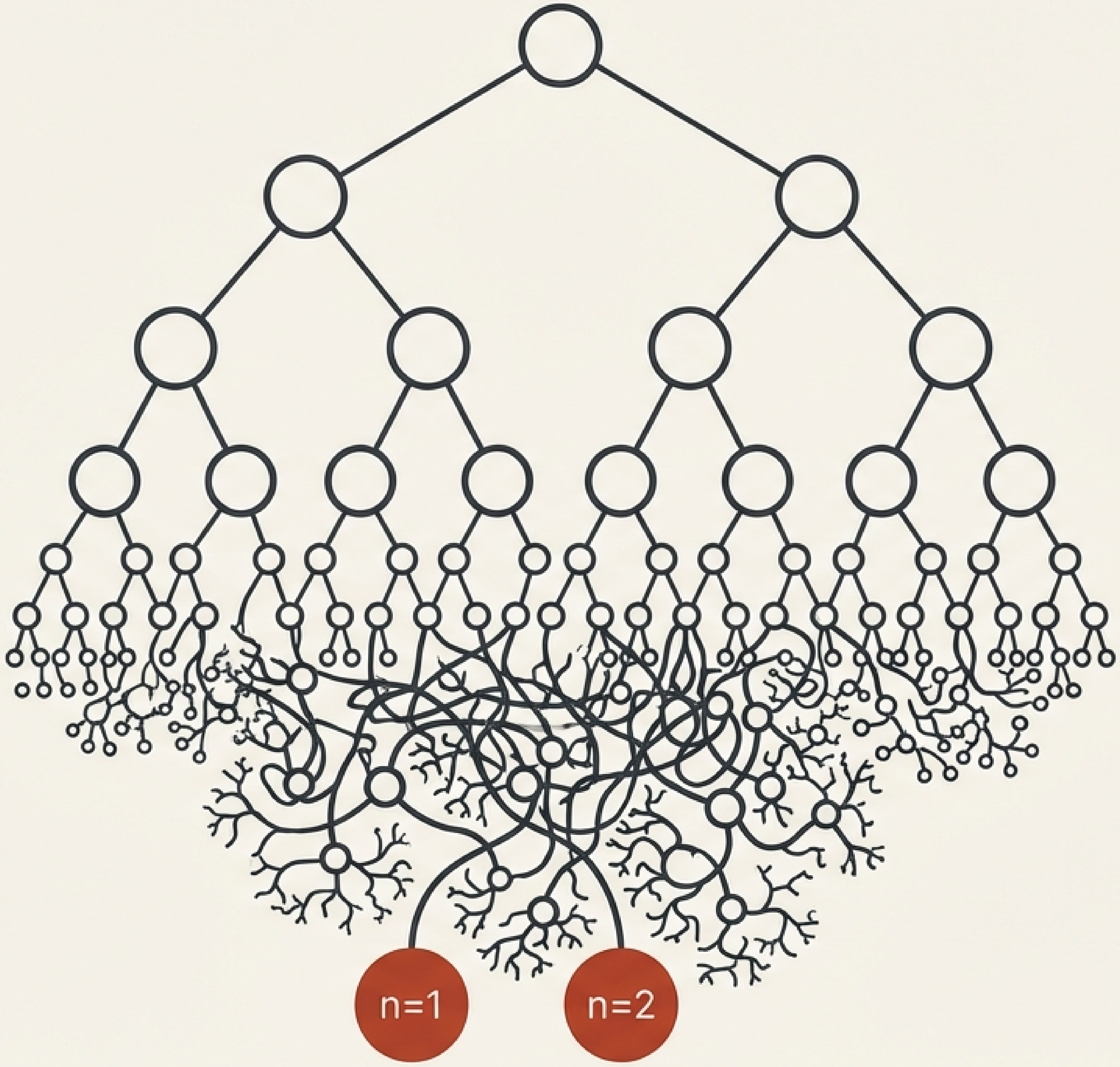
Taming Decision Trees

A visual guide to hyperparameter tuning and model complexity. Moving from theoretical geometry to actionable Scikit-Learn application.





The Inherent Flaw of the Greedy Algorithm

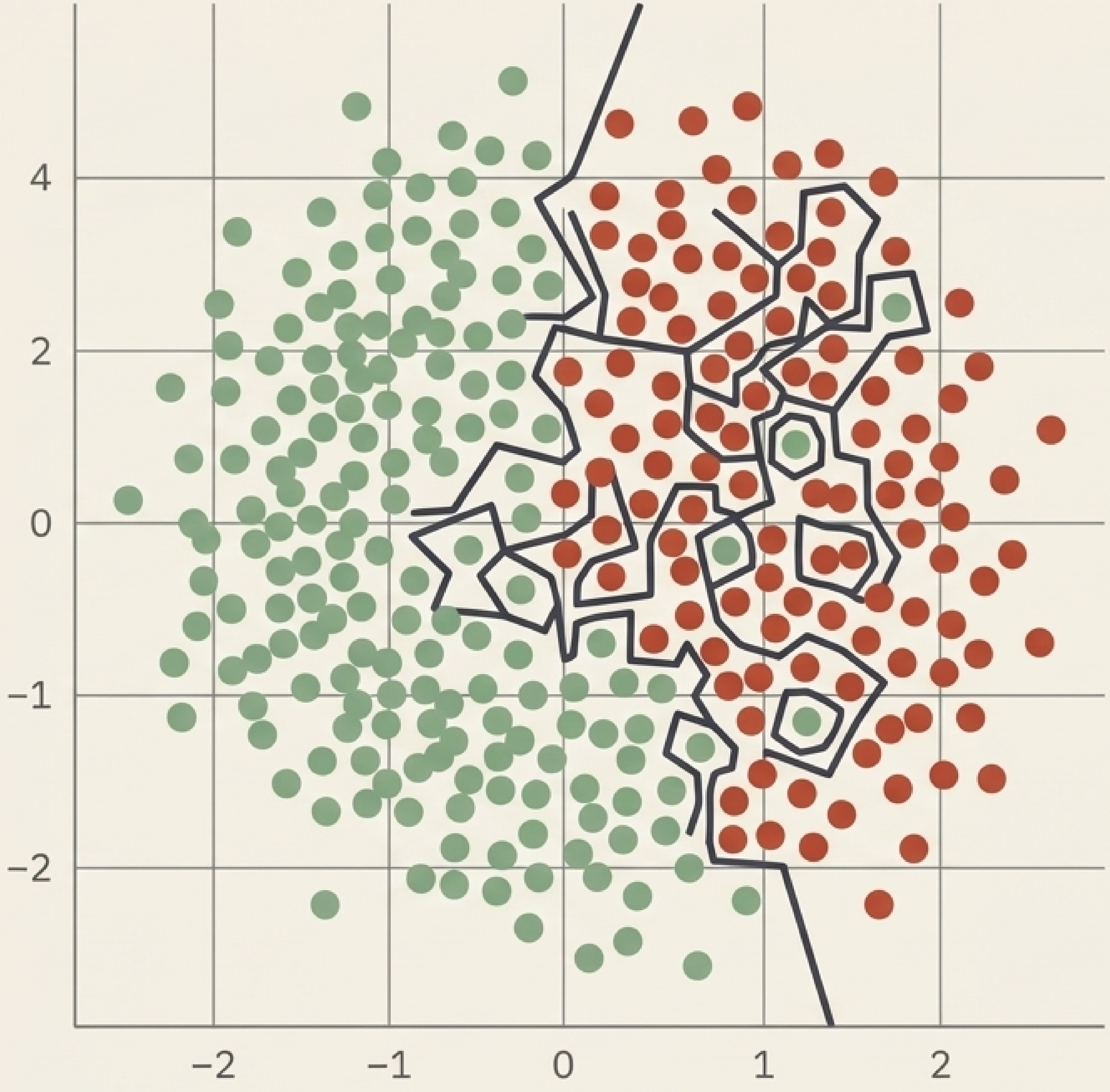
- Decision trees are inherently greedy.
- Without intervention, they continue splitting data until every leaf node is perfectly pure.
- The Result: The model memorises the training data, capturing random noise rather than the true underlying pattern.

Visualising Overfitting (High Variance)

The symptom: Flawless training accuracy, terrible real-world performance.

The cause: The tree has drawn hyper-specific boundaries around isolated data points.

A fully grown tree (e.g., `max_depth=None`) inevitably leads to this state.



Visualising Underfitting (High Bias)

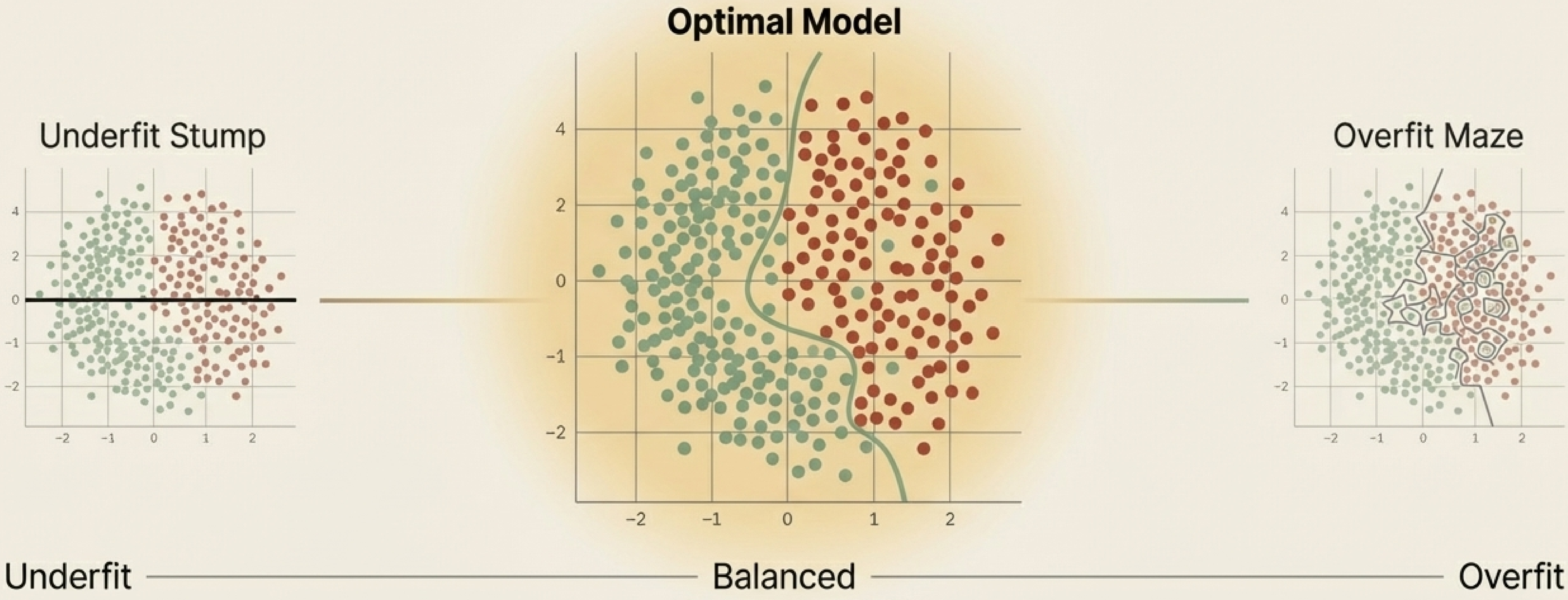
The symptom: Poor accuracy across both training and test data.

The cause: The tree was restricted too heavily (a stump).

Example: Setting `max_depth=1` creates a model too rigid to understand complex data relationships.

The Goal is the Goldilocks Zone

- Hyperparameter tuning is the art of finding the perfect middle ground.
- We use Scikit-Learn’s built-in parameters as dials to shape the decision boundaries physically.



Core Splitting Logic

Parameter: `criterion`

`gini` vs `entropy`

Rule of Thumb: Gini is generally preferred and slightly faster. The performance difference is typically minimal (often <1%).

Parameter: `splitter`

`best` vs `random`

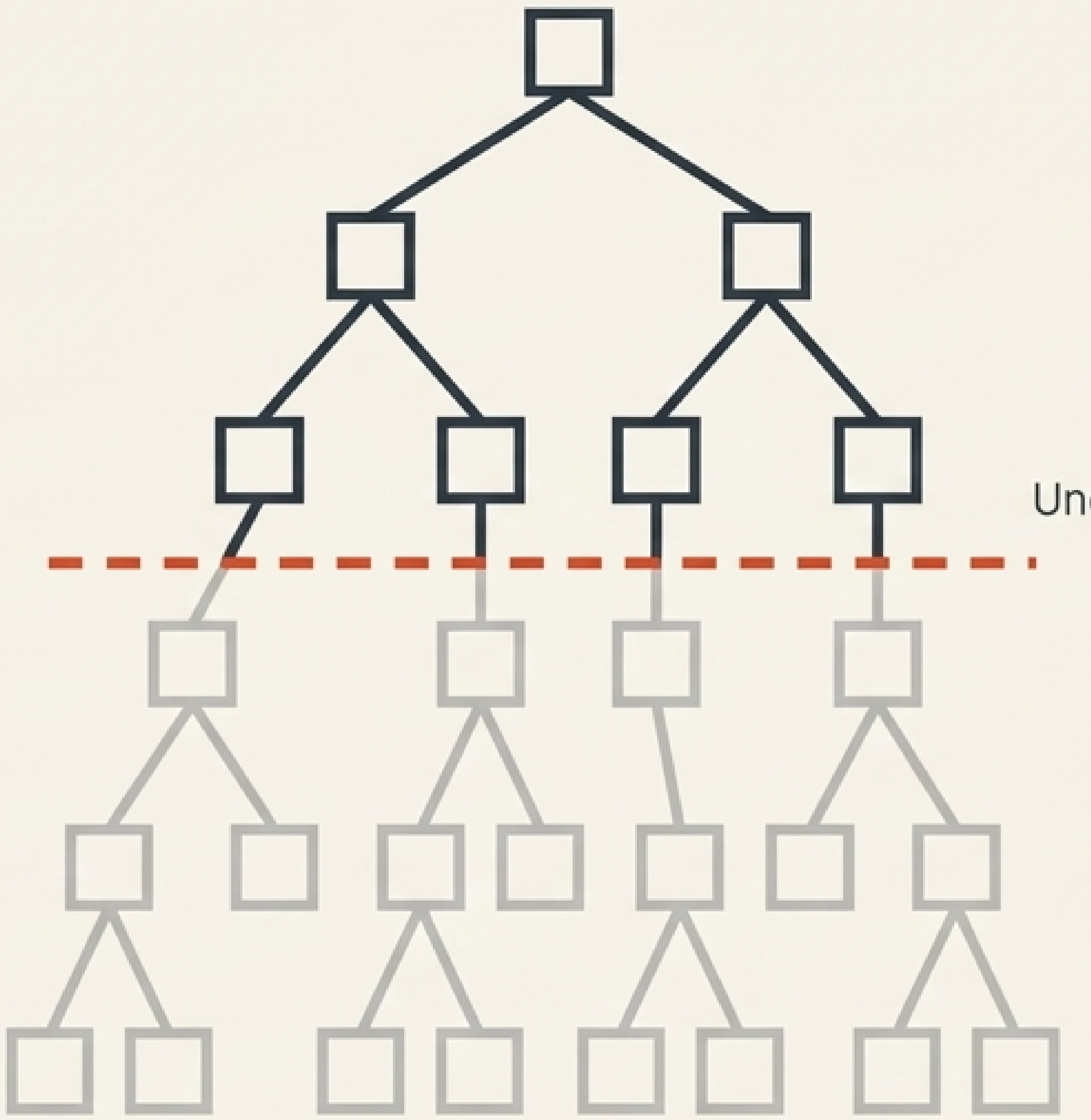
Rule of Thumb: ‘best’ evaluates all splits; ‘random’ introduces variance. Choosing ‘random’ is an effective, simple way to reduce overfitting.

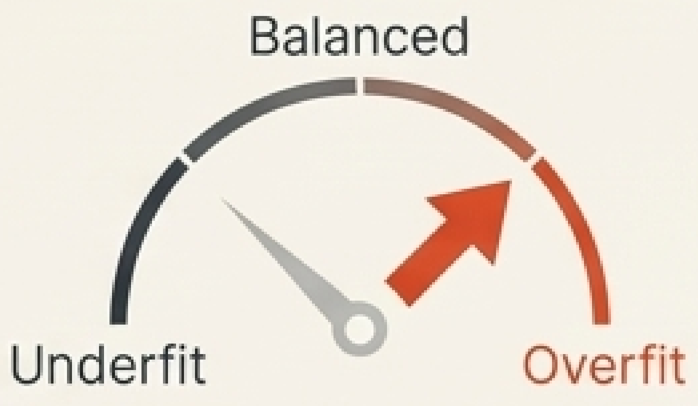
Controlling Overall Tree Height

Parameter: `max_depth`

Sets a hard limit on how many levels the tree can grow.

Dial Check:
Low value = Underfitting risk.
High value (or default `None`) = Overfitting risk.





Restricting Internal Branches

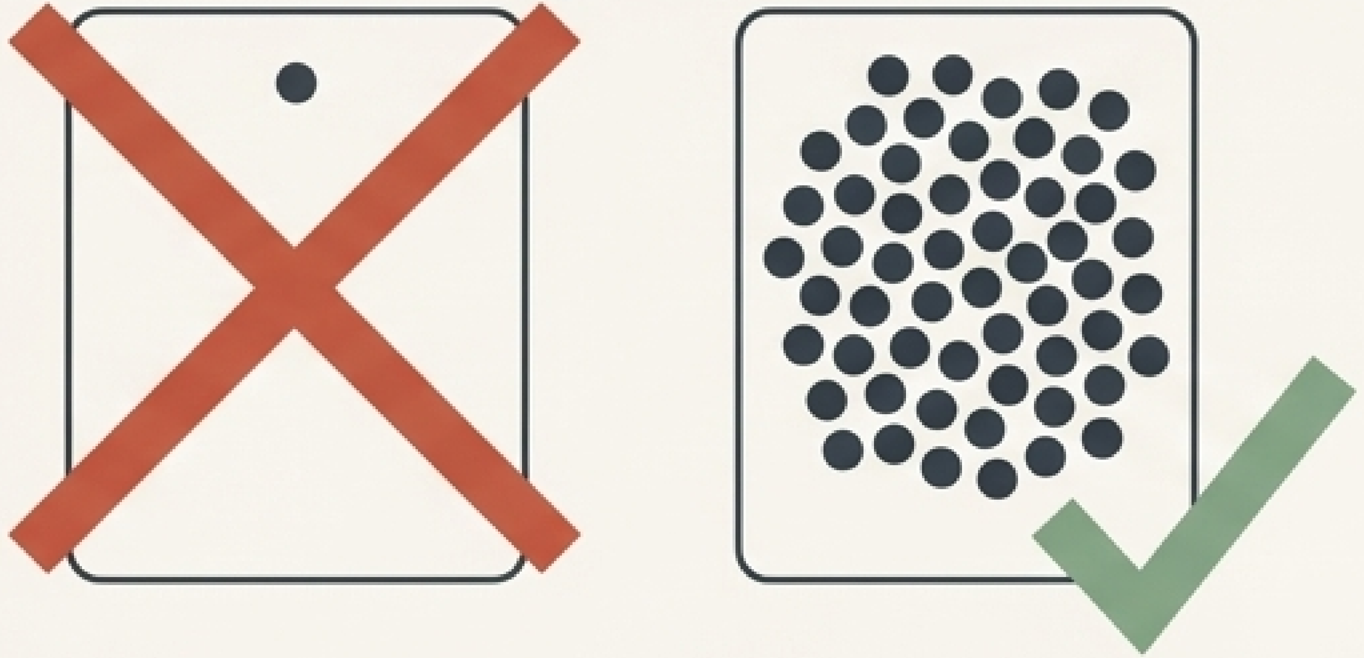
Parameter: `min_samples_split`

- Dictates the minimum number of data points required to justify creating a new branch.
- Prevents the algorithm from dividing tiny clusters of data into hyper-specific rules.

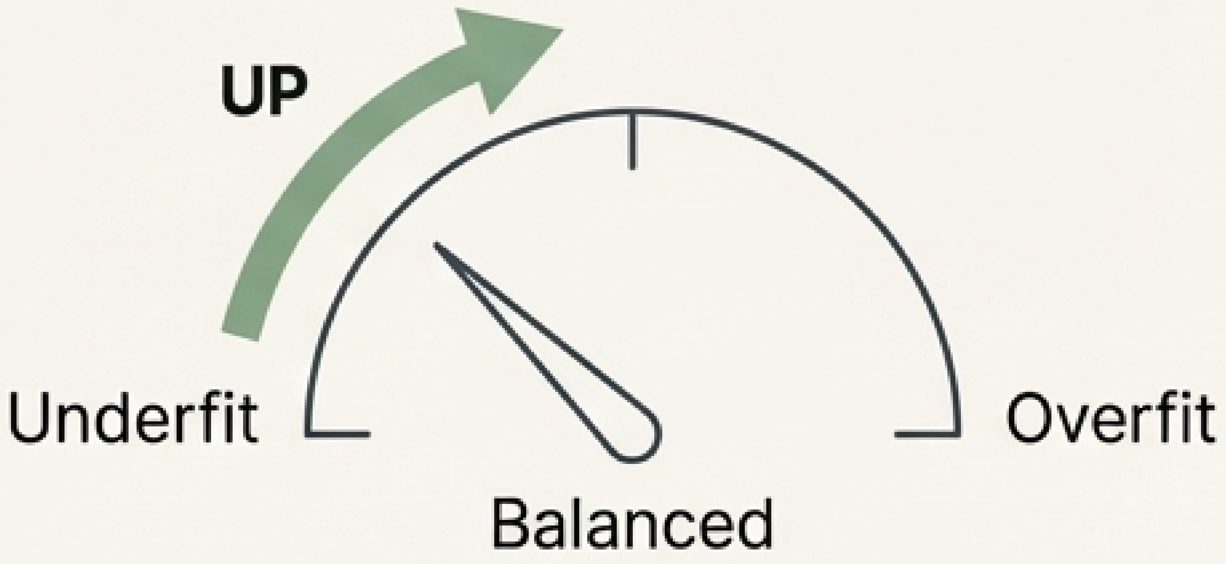
Enforcing Evidence at the Leaves

Parameter: `min_samples_leaf`

Ensures every final category (leaf) contains a minimum baseline of data points.



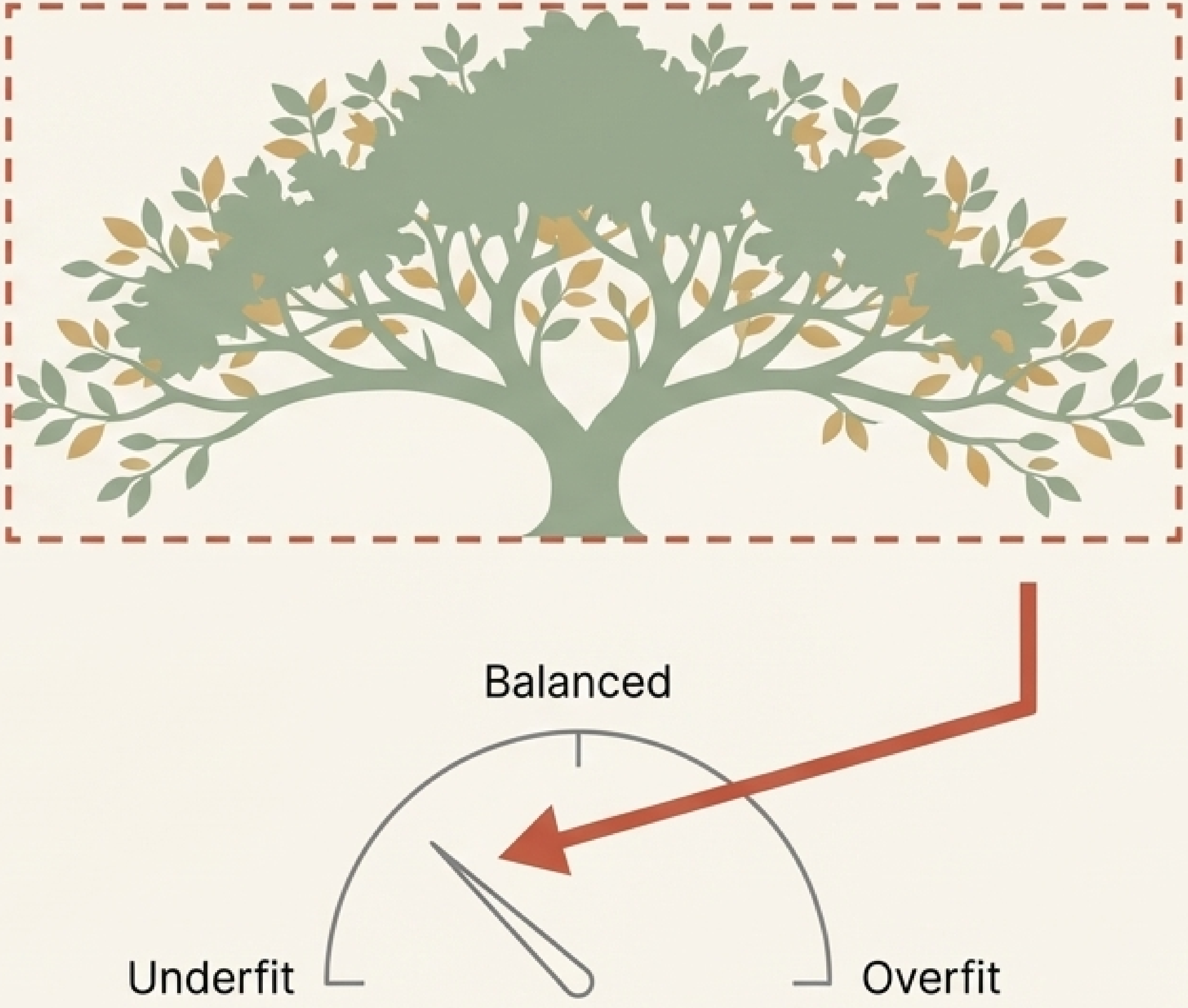
Rule of Thumb: Extremely effective at smoothing out decision boundaries and and preventing isolated, single-point conclusions.



Limiting Total Endpoints

Parameter: `max_leaf_nodes`

- A direct cap on the total number of leaves the entire tree can generate.
- Forces the algorithm to prioritise only the most mathematically important splits, discarding minor local optimizations.



Restricting the Algorithm's Vision

Parameter: `max_features`

Restricts the number of columns the tree is allowed to look at during any given split.

Rule of Thumb: By preventing the tree from relying on the same dominant features repeatedly, you force it to find alternative patterns. This heavily reduces overfitting.

Pruning by Required Return on Investment

Parameter: `min_impurity_decrease`

Sets a minimum threshold for improvement. A split will only execute if it improves the purity of the data by this exact margin or more. Acts.

Acts as an early stopping mechanism based on mematical value rather than just depth or sample counts.

The Practitioner's Cheat Sheet

Keep this matrix as your primary reference when tuning Scikit-Learn models.

To Reduce Overfitting (Decrease Model Complexity):	
Decrease: `max_depth` `max_features`, `max_leaf_nodes`	Increase: `min_samples_split` `min_samples_leaf` `min_impurity_decrease`
To Reduce Underfitting (Increase Model Complexity):	
Reverse the above.	

The Foundation for Advanced Architecture

A single tuned tree is just the beginning.

Mastering these exact Scikit-Learn parameters is the absolute prerequisite for building robust Random Forests and Gradient Boosting architectures.

Control the tree to build the forest.